Document number: P1466r1

Howard E. Hinnant
2019-03-10

Miscellaneous minor fixes for chrono

Contents

Introduction

This is a collection of minor fixes and upgrades to the `<chrono>` library that have come to my attention since the acceptance of P0355r7.

Wording

1. Feedback from the field asks for a way to identify if a `utc_time` represents a leap second. This proposed API is the only API that has field experience in providing that information in a way that is efficient, and provides all information the client desires when making a query in this area. It is used in the implementation of the conversion between `utc_time` and `sys_time`, and in the parsing and formatting of `utc_time`, so is likely to be present in any event. If we choose to specify it, it can be spelled without underscores, and be available to clients of `<chrono>`.

   Insert into the synopsis 26.2 Header `<chrono>` synopsis [time.syn]:

   ```
   struct leap_second_info;

   template <class Duration>
      leap_second_info get_leap_second_info(utc_time<Duration> const& ut);
   ```

   Insert new paragraphs in 26.7.2.3 Non-member functions [time.clock.utc.nonmembers]:

   ```
   struct leap_second_info
   {
       bool    is_leap_second;
       seconds elapsed;
   };
   ```

   The type `leap_second_info` has data members, and special members specified above. It has no base classes or members other than those specified.

   ```
   template <class Duration>
   leap_second_info
   get_leap_second_info(utc_time<Duration> const& ut);
   ```

   Returns: A `leap_second_info` where `is_leap_second` is true if `ut` is during a leap second insertion, and otherwise false. `elapsed` is the number of leap seconds between 1970-01-01 and `ut`. If `is_leap_second` is true, the leap second referred to by `ut` is included in the count.

2. Feedback from the field asks for a way to customize the spacing between a duration's value and its unit (e.g. supply a space, or a Unicode custom space between the value and the unit). This item proposes `%Q` to represent the durations's value and `%q` to represent the duration's unit, for formatting only. Example: `format("%Q %q", 45ms) == "45 ms"`.

Add two new rows to Table 87 — Meaning of `format` conversion specifiers:

`%Q`	The duration's numeric value (as if extracted via `.count()`).
`%q`	The duration's unit suffix as specified in [time.duration.io].

3. About half of the clients are upset that the currently specified encoding for `weekday` implies that Sunday is the first day of the week (consistent with the current C and C++ specifications for `tm.tm_wday`), and the other half of the clients will be upset if the `weekday` encoding follows the ISO specification of [1, 7] maps to [Monday, Sunday].

   This change strikes a compromise in an attempt to please everyone (a nearly impossible task).

   - The `weekday{unsigned}` constructor accepts both mappings, which means that [0, 6] maps to [Sunday, Saturday] **and** [1, 7] maps to [Monday, Sunday]. This is possible by simply accepting [0, 7] where [1, 6] maps to [Monday, Saturday] and both 0 and 7 map to Sunday.
   - The explicit conversion to `unsigned` is removed from `weekday` and named conversions are inserted in its place: `c_encoding()` and `iso_encoding()`. The client can choose which mapping from `weekday` to `unsigned` he desires by choosing one of these member functions.

Modify 26.8.6.2 [time.cal.wd.members] as indicated:

```
constexpr explicit weekday(unsigned wd) noexcept;
```

Effects: Constructs an object of type `weekday` by initializing `wd_` with `wd`, except if `wd_ == 7`, stores `0`. The value held is unspecified if wd is not in the range [0, 255].

~~`constexpr explicit operator unsigned() const noexcept;`~~

~~*Returns:* `wd_`.~~

`constexpr unsigned c_encoding() const noexcept;`

Returns: `wd_`.

`constexpr unsigned iso_encoding() const noexcept;`

Returns: `wd_ == 0u ? 7u : wd_`.

4. `time_of_day` is poorly specified. It suffers from an overly complicated specification, and at the same time, insufficient functionality from that specification. Additionally, guidance from the LEWG mailing list indicates significant dissatisfaction with the name of `time_of_day` and its internal state that changes mode between 24h and 12h time. This note changes the name of `time_of_day` to `hh_mm_ss` to bring the semantics away from holding just a time duration since midnight, and more towards holding *any* duration as an `{hours, minutes, seconds, subseconds}` data structure. Also the 24h/12h functionality is moved out of this structure and into namespace-scope functions. And finally this change both

simplifies the specification while providing more pertinent functionality. However the basic functionality is maintained: `hh_mm_ss` is an `{hours, minutes, seconds, subseconds}` structure which easily converts to and from a `duration` and provides easy formatting.

Modify 26.1 General [time.general]

Table 85 — Time library summary

	Subclause	**Header(s)**
...		
26.8	Civil calendar	
26.9	Class template ~~`time_of_day`~~`hh_mm_ss`	
26.10	12/24 hour functions	
~~26.10~~26.11	Time zones	
...		

Modify 26.2 Header `<chrono>` synopsis [time.syn]

```
...
// 26.9, class template time_of_dayhh_mm_ss
template <class Duration> class time_of_dayhh_mm_ss;
template<> class time_of_day<hours>;
template<> class time_of_day<minutes>;
template<> class time_of_day<seconds>;
template<class Rep, class Period> class time_of_day<duration<Rep, Period>>;

template<class charT, class traits>
  basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const time_of_day<hours>& t);
template<class charT, class traits>
  basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const time_of_day<minutes>& t);
template<class charT, class traits>
  basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const time_of_day<seconds>& t);
template<class charT, class traits, class Rep, class PeriodDuration>
  basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os,
               const time_of_dayhh_mm_ss<Dduration<Rep, Period>>& t);

// 26.10, 12/24 hour functions
constexpr bool is_am(hours const& h) noexcept;
constexpr bool is_pm(hours const& h) noexcept;
constexpr hours make12(hours h) noexcept;
constexpr hours make24(hours h, bool is_pm) noexcept;
...
```

Modify 26.7.1.3 Non-member functions [time.clock.system.nonmembers]

Effects:

...

```
os << year_month_day{dp} << ' ' << time_of_dayhh_mm_ss{tp-dp};
```

Replace [time.tod] with [time.hms]:

26.9 Class template `hh_mm_ss` [time.hms]

26.9.1 Overview [time.hms.overview]

```
template <class Duration>
class hh_mm_ss
{
    bool            is_neg; // exposition only
    chrono::hours   h;      // exposition only
    chrono::minutes m;      // exposition only
    chrono::seconds s;      // exposition only
    precision       ss;     // exposition only

public:
    static unsigned constexpr fractional_width = see below;
    using precision                            = see below;

    constexpr hh_mm_ss() noexcept : hh_mm_ss{Duration::zero()} {}
    constexpr explicit hh_mm_ss(Duration d) noexcept;

    constexpr bool is_negative() const noexcept;
    constexpr chrono::hours hours() const noexcept;
    constexpr chrono::minutes minutes() const noexcept;
    constexpr chrono::seconds seconds() const noexcept;
    constexpr precision subseconds() const noexcept;

    constexpr explicit operator  precision()   const noexcept;
    constexpr          precision to_duration() const noexcept;
};

template <class charT, class traits, class Duration>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, hh_mm_ss<Duration> const& hms);
```

The `hh_mm_ss` class template splits a duration into a “broken down” time *hours:minutes:seconds* and possibly *subseconds*, where *subseconds* will be a `duration` unit based on a negative power of 10. The `Duration` template parameter dictates the precision to which the time is broken down. A `hh_mm_ss` models negative durations with a distinct `is_negative` getter that returns `true` when the input duration is negative. The individual duration fields always return non-negative durations even when `is_negative()` indicates the structure is representing a negative duration.

If `Duration` is not an instance of `duration`, the program is ill-formed.

26.9.2 Members [time.hms.members]

```
static unsigned constexpr fractional_width = see below;
```

`fractional_width` is the number of fractional decimal digits represented by `precision`. `fractional_width` has the value of the smallest possible integer in the range [0, 18] such that `precision` will exactly represent all values of `Duration`. If no such value of `fractional_width` exists, then `fractional_width` is 6.

[*Example:*

`Duration`	**`fractional_width`**	**Formatted fractional second output of `Duration{1}`**

`hours`, `minutes`, and `seconds`	`0`	
`milliseconds`	`3`	.001
`microseconds`	`6`	.000001
`nanoseconds`	`9`	.000000001
`duration<int, ratio<1, 2>>`	`1`	.5
`duration<int, ratio<1, 3>>`	`6`	.333333
`duration<int, ratio<1, 4>>`	`2`	.25
`duration<int, ratio<1, 5>>`	`1`	.2
`duration<int, ratio<1, 6>>`	`6`	.166666
`duration<int, ratio<1, 7>>`	`6`	.142857
`duration<int, ratio<1, 8>>`	`3`	.125
`duration<int, ratio<1, 9>>`	`6`	.111111
`duration<int, ratio<1, 10>>`	`1`	.1
`duration<int, ratio<756, 625>>` // *microfortnights*	`4`	.2096

— *end example*]

`using precision = see below;`

`precision` is `duration<common_type_t<Duration::rep, seconds::rep>, ratio<1, 10`$^{\texttt{fractional\_width}}$`>>`.

`constexpr explicit hh_mm_ss(Duration d) noexcept;`

Effects: constructs an object of type `hh_mm_ss` which represents the `Duration d` with precision `precision`. The `hours`, `minutes`, `seconds` and `subseconds` are stored. Also stored is the fact if `d` is negative or non-negative.

Stores `d < Duration::zero()` in `is_neg`.

Stores `duration_cast<chrono::hours>(abs(d))` in `h`.

Stores `duration_cast<chrono::minutes>(abs(d) - hours())` in `m`.

Stores `duration_cast<chrono::seconds>(abs(d) - hours() - minutes())` in `s`.

If `treat_as_floating_point_v<precision::rep>` is `true`, stores `abs(d) - hours() - minutes() - seconds()` in `ss`.

Else stores `duration_cast<precision>(abs(d) - hours() - minutes() - seconds())` in `ss`. [*Note:* When `precision` is `seconds` with integral representation, `subseconds()` always returns `0s` — *end note*]

Ensures: If `treat_as_floating_point_v<precision::rep>` is `true`, `to_duration()` returns `d`, else `to_duration()` returns `floor<precision>(d)`.

```
constexpr bool is_negative() const noexcept;
```

Returns: `is_neg`.

```
constexpr chrono::hours hours() const noexcept;
```

Returns: `h`.

```
constexpr chrono::minutes minutes() const noexcept;
```

Returns: `m`.

```
constexpr chrono::seconds seconds() const noexcept;
```

Returns: `s`.

```
constexpr precision subseconds() const noexcept;
```

Returns: `ss`.

```
constexpr precision to_duration() const noexcept;
```

Returns: If `is_neg`, returns `-(h + m + s + ss)`, else returns `h + m + s + ss`.

```
constexpr explicit operator precision() const noexcept;
```

Returns: `to_duration()`.

26.9.3 Non-members [time.hms.nonmembers]

```
template <class charT, class traits, class Duration>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, hh_mm_ss<Duration> const& hms);
```

Effects: Outputs to `os` according to the format `"%T"` ([time.format]). If `hms.is_negative()`, the output is preceded with `'-'`.

Returns: `os`.

[*Example:*

```
for (auto ms : {-4083007ms, 4083007ms, 65745123ms})
{
     hh_mm_ss hms{ms};
     cout << hms << '\n';
}
cout << hh_mm_ss{65745s} << '\n';
```

Produces the output (assuming the "C" locale):

```
-01:08:03.007
01:08:03.007
18:15:45.123
18:15:45
```

— end example]

Insert a new section, 26.10 12/24 hour functions [time.12]:

26.10 12/24 hour functions [time.12]

These functions aid in translating between a 12h format time of day, and a 24h format time of day.

```
constexpr bool is_am(hours const& h) noexcept;
```

Returns: `0h <= h && h <= 11h`.

```
constexpr bool is_pm(hours const& h) noexcept;
```

Returns: `12h <= h && h <= 23h`.

```
constexpr hours make12(hours h) noexcept;
```

Returns: The 12-hour equivalent of `h` in the range `[1h, 12h]`. If `h` is not in the range `[0h, 23h]`, the value returned is unspecified.

```
constexpr hours make24(hours h, bool is_pm) noexcept;
```

Returns: If `is_pm` is `false`, returns the 24-hour equivalent of `h` in the range `[0h, 11h]`, assuming `h` represents an ante meridiem hour. Else returns the 24-hour equivalent of `h` in the range `[12h, 23h]`, assuming `h` represents a post meridiem hour. If `h` is not in the range `[1h, 12h]`, the value returned is unspecified.